

Table des matières

Introduction	2
1 Cadre formel de la mesure de dégradation	4
1.1 Systèmes	4
1.2 Compteurs	7
1.2.1 Compteur exact	8
1.2.2 Compteur approché	8
2 Temps de détection d'une faute	10
2.1 Définitions	10
2.2 Temps minimal et maximal	11
3 Cycles et paire de cycles ambiguë	17
3.1 Définitions	17
3.2 Paire de cycles ambiguë et fonction de comptage	18
3.3 Cycles concurrents et fonction de comptage additive	20
4 Construction par recouvrement de compteurs approchés	24
4.1 Cadre formel	24
4.2 Exemples d'application	25
4.3 Limites	29
4.3.1 Optimalité du recouvrement	29
4.3.2 Dégradation de l'estimation	34
5 Conclusion	36
Références	38

Introduction

Aujourd'hui, nous vivons dans un monde de plus en plus assisté par la technologie. Des objets du quotidien comme nos smartphones, voitures, ou appareils électroménagers intègrent maintenant des **systèmes techniques et logiciels** toujours plus complexes. Seulement, dans ces systèmes, la **dégradation** est une chose inévitable qui a pour cause différents phénomènes tels que l'usure des composants physiques, l'obsolescence des technologies ou encore des interactions imprévues avec l'environnement. Ces dégradations finissent souvent par entraîner des anomalies, des dysfonctionnements voire des pannes qui, dans les cas les plus graves, peuvent causer des dommages matériels considérables ou encore mettre en péril la sécurité des utilisateurs. Il est donc évident que l'étude ainsi que la capacité à comprendre, détecter et mesurer cette dégradation devient un enjeu majeur dans le développement de nos systèmes techniques et informatiques afin de garantir leur fiabilité et leur sécurité. Pour atteindre ce but, diverses techniques telles que le **model-checking** sont aujourd'hui mises en place.

Le model-checking désigne une méthode de vérification formelle et automatique d'un système qui permet de confirmer que ce dernier satisfait bien des propriétés voulues. Pour ce faire, les systèmes sont souvent modélisés à l'aide d'automates, c'est-à-dire des graphes orientés dans lesquels un sommet représente un état du système et un arc représente une transition entre ces états. Il convient toutefois de noter que les automates ne constituent qu'un des nombreux types de modèles utilisés en model-checking ; selon la nature des systèmes et des propriétés à vérifier, il est possible de recourir à d'autres formalismes. Il faut cependant garder en tête que le model-checking demeure un défi, notamment car ces modèles doivent prendre en compte la possibilité que plusieurs défaillances (ou **fautes**) aient lieu l'une après l'autre ou même simultanément, ce qui complexifie l'analyse et la vérification des propriétés du système.

Parmi les directions explorées dans ce domaine figure notamment le **diagnostic**, qui vise à détecter la survenue de fautes. Bien que cette approche soit précieuse pour assurer la détection de comportements anormaux, elle est toutefois limitée, reposant sur une distinction binaire entre la présence et l'absence de fautes, sans tenir compte de leur gravité, de la fréquence ou de l'impact de celles-ci sur le fonctionnement des systèmes. Cette limitation rend difficile une évaluation fine de la dégradation, qui nécessite alors des outils capables de quantifier ou de hiérarchiser les défaillances au-delà d'une simple détection.

L'objectif de ce travail, mené dans le cadre du module d'initiation à la recherche, a été d'étudier des approches formelles permettant la représentation de systèmes, et d'examiner des méthodes de détection et d'estimation de fautes à partir de leurs exécutions. Ce rapport vise à présenter une démarche progressive allant de la formalisation des notions utilisées à l'élaboration de mécanismes concrets d'estimation.

Dans un premier temps, il convient de poser les fondements théoriques nécessaires à l'étude, en définissant formellement les systèmes que nous avons considérés ainsi que les compteurs utilisés pour évaluer la dégradation (section 1). Sur cette base, nous avons décidé d'axer notre analyse sur l'étude du temps de détection d'une faute (section 2), l'identification de certaines limites structurelles impactant les compteurs, notamment à travers l'étude des cycles et des paires de cycles ambiguës (section 3), avant de finalement proposer notre méthode de construction pour les compteurs (section 4).

1 Cadre formel de la mesure de dégradation

La mesure de la dégradation d'un système vise à évaluer l'état de détérioration d'un système au cours de son exécution. Dans de nombreux cas, les fautes, au sens des déviations de comportement par rapport à celui qui est attendu, ne sont pas directement observables. Il est alors nécessaire de recourir à des méthodes d'estimation basées sur les traces d'exécution disponibles.

Il s'agit donc d'utiliser les informations observables d'une exécution du système pour déduire une estimation du nombre de fautes s'étant produites, et ce sans disposer d'une visibilité explicite sur ces mêmes fautes. Avec cette approche, il est possible d'associer un niveau de dégradation à un système à partir de l'une de ses exécutions.

Dans le cadre de ce rapport, nous avons décidé de réaliser notre analyse sur des systèmes représentés par des systèmes de transition d'états. Ce modèle, caractérisé par un ensemble d'états et de transitions étiquetées, constitue un formalisme de base pour la modélisation du comportement de systèmes et nous a été inspiré par l'article "A Tale of Two Diagnoses in Probabilistic Systems" [1]. Voici les définitions que nous proposons.

1.1 Systèmes

Ici, nous définissons formellement le type de modèle utilisé, ainsi que les différents comportements qu'il produit.

Définition 1 (Système de transition). *Un système de transition $\mathcal{A} = (Q, \Sigma, \rightarrow, q_0)$ est défini comme suit :*

- Q : l'ensemble des états du système,
- $\Sigma = \Sigma_o \cup \Sigma_u$: l'ensemble des étiquettes de transition,
- $\rightarrow (\subseteq Q \times \Sigma \times Q)$: une relation de transition,
- $q_0 \in Q$: l'état initial du système.

où Σ_o est l'ensemble des étiquettes des transitions observables, Σ_u l'ensemble des étiquettes des transitions inobservables et $f \in \Sigma_u$ est l'étiquette des transitions dites fautives.

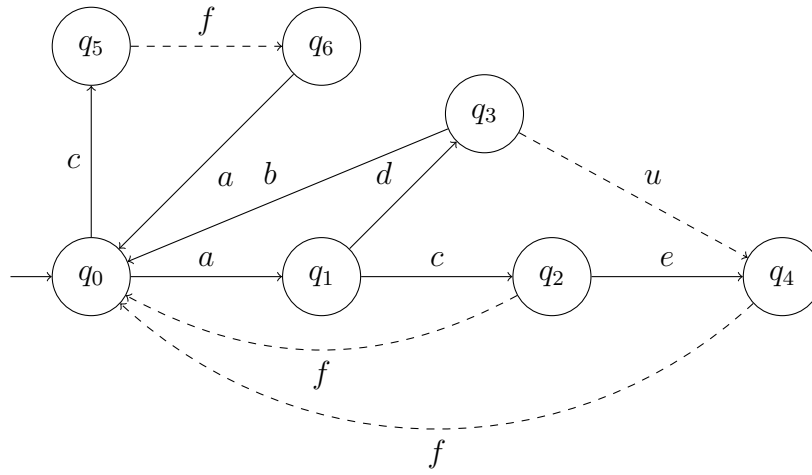


FIGURE 1 – Exemple d'un système de transition

Visuellement, les transitions étiquetées par des étiquettes de Σ_o sont des flèches pleines, et celles étiquetées par des étiquettes de Σ_u des flèches pointillées.

Afin de représenter les comportements possibles d'un système, nous utilisons les notions de séquences et d'exécutions, que nous distinguons comme suit.

Définition 2 (Séquence et exécution). *Une séquence γ de $\mathcal{A}$ est une suite de transitions, de la forme :*

$$\gamma = q'_0 \xrightarrow{\sigma_1} q_1 \xrightarrow{\sigma_2} \dots \xrightarrow{\sigma_n} q_n \text{ avec } \sigma_i \in \Sigma$$

et telle que pour chaque transition $q \xrightarrow{\sigma} q'$ de γ , $(q, \sigma, q') \in \rightarrow$.

Une séquence ρ de $\mathcal{A}$ est une exécution de $\mathcal{A}$ si et seulement si l'état de départ de la première transition est l'état initial de $\mathcal{A}$.

Notation. *Le nombre de fautes dans une séquence γ est noté $|\gamma|_f$*

Dans le cas du système de transition de la figure 1, une séquence γ et une exécution ρ pourraient être :

$$\begin{aligned} \sigma &= q_3 \xrightarrow{u} q_4 \xrightarrow{f} q_0 \xrightarrow{a} q_1 \xrightarrow{d} q_3 \\ \rho &= q_0 \xrightarrow{a} q_1 \xrightarrow{d} q_3 \xrightarrow{b} q_0 \xrightarrow{a} q_1 \xrightarrow{c} q_2 \xrightarrow{e} q_4 \xrightarrow{f} q_0 \xrightarrow{a} q_1 \end{aligned}$$

Dans le cadre de notre modèle, les séquences et les exécutions donnent lieu à des traces, c'est-à-dire une suite d'étiquettes correspondant à des événements observables et inobservables.

Définition 3 (Trace). *Pour une séquence $\gamma = q_0 \xrightarrow{\sigma_1} q_1 \xrightarrow{\sigma_2} \dots \xrightarrow{\sigma_{n-1}} q_{n-1} \xrightarrow{\sigma_n} q_n$*

La trace $\tau_\gamma \in \Sigma$ produite à partir de $\gamma \in \rightarrow^$ est définie comme suit :*

$$\tau_\gamma = \sigma_1 \cdot \sigma_2 \cdot \dots \cdot \sigma_{n-1} \cdot \sigma_n$$

Toutefois, dans la pratique, seuls les événements observables sont accessibles à un observateur. Il est donc nécessaire d'introduire la notion d'observation, qui correspond l'extraction des étiquettes des événements observables d'une exécution, et par extension d'une trace. Une observation est donc une suite d'étiquettes issues uniquement des événements observables d'un système, et elle permet de décrire ce qui est perçu depuis l'extérieur sans accès aux comportements internes.

Définition 4 (Observation). *Pour une séquence $\gamma = q_0 \xrightarrow{\sigma_1} q_1 \xrightarrow{\sigma_2} \dots \xrightarrow{\sigma_{n-1}} q_{n-1} \xrightarrow{\sigma_n} q_n$*

L'observation $\pi_\gamma \in \Sigma_o^$ produite à partir de $\gamma \in \rightarrow^*$ est définie inductivement comme suit :*

Cas de base : $\pi_\epsilon = \epsilon$, où ϵ est une séquence vide qui produit une observation vide,

Cas général : Pour γ et $t = (q_n, \sigma, q') \in \rightarrow$, on a :

$$\pi_{\gamma \cdot t} = \begin{cases} \pi_\gamma \cdot \sigma & \text{si } \sigma \in \Sigma_o \\ \pi_\gamma & \text{si } \sigma \in \Sigma_u \end{cases}$$

À partir de γ et ρ , définies plus tôt, on peut obtenir les traces et les observations suivantes :

$$\begin{aligned} \tau_\gamma &= ufad \text{ et } \pi_\gamma = ad \\ \tau_\rho &= adbacefa \text{ et } \pi_\rho = adbacea \end{aligned}$$

1.2 Compteurs

Les systèmes et leurs différents comportements définis, nous expliquons maintenant comment mesurer la dégradation. Nous introduisons pour cela la notion de compteurs, fonctions définies sur les observations des systèmes et permettant d'estimer de le nombre de fautes s'étant produites lors d'exécutions.

Définition 5 (Compteur). *Un compteur renvoie une estimation du nombre de faute dans une exécution à partir de l'observation associée, et est une fonction telle que :*

$$C : \Sigma_o^* \longrightarrow \mathbb{N}$$

De cette manière, nous obtenons $C(\pi_\rho)$ une estimation du nombre de fautes dans une exécution ρ de $\mathcal{A}$.

Pour être pertinent, un compteur doit respecter certaines propriétés fondamentales en lien avec le nombre réel de fautes $|\rho|_f$ dans une exécution ρ d'un système $\mathcal{A}$. Ces propriétés sont les suivantes :

- $C(\pi_\rho) \leq |\rho|_f$: le compteur ne doit jamais surestimer le nombre réel de fautes,
- il existe une fonction g telle que $|\rho|_f \leq g(C(\pi_\rho))$: le nombre réel de fautes est borné par une fonction dépendant de la valeur donnée par le compteur.

Ces propriétés permettent de garantir que le compteur fournit une estimation cohérente et encadrée du nombre de fautes, tout en laissant la possibilité de définir différents types de compteurs selon les besoins et les contextes.

Pour l'observation $\pi_\rho = adbacea$ obtenue plus tôt, nous aurions par exemple un compteur C donnant le résultat $C(\pi_\rho) = 1$.

1.2.1 Compteur exact

Pour toute exécution ρ de $\mathcal{A}$, un compteur exact satisfait la propriété suivante :

$$C(\pi_\rho) = |\rho|_f$$

Ce type de compteur suppose que toutes les fautes survenant au cours d'une exécution sont comptées dans l'estimation, ce qui revient à disposer d'une visibilité totale du système et donc d'une connaissance immédiate des fautes. Une telle hypothèse est cependant peu réaliste dans la mesure où les fautes sont, par nature, inobservables. L'utilisation de ce type de compteur est donc impossible, et est donc à exclure.

1.2.2 Compteur approché

Un compteur approché vise à estimer le nombre de fautes dans une exécution, sans exiger une correspondance exacte avec la réalité. Il fournit un minorant N du nombre réel de fautes, tout en autorisant une certaine marge d'erreur liée au compteur.

Nous distinguerons parmi ces fonctions différents types de marges, en fonction des propriétés des systèmes étudiés.

Marge d'erreur additive

Un compteur approché additif permet l'omission d'au plus K erreurs dans l'estimation qu'il produit, satisfaisant ainsi la propriété suivante pour toute exécution ρ d'un système $\mathcal{A}$:

$$C(\pi_\rho) \leq |\rho|_f \leq C(\pi_\rho) + K$$

Marge d'erreur multiplicative

Un compteur approché multiplicatif permet une erreur proportionnelle à l'estimation à partir d'un facteur multiplicatif K_1 et d'une constance additive K_2 , satisfaisant la propriété suivante pour toute exécution ρ d'un système $\mathcal{A}$:

$$C(\pi_\rho) \leq |\rho|_f \leq K_1 \times C(\pi_\rho) + K_2$$

Le coefficient K_1 prend en charge l'incertitude due à l'existence de différentes séquences pouvant produire une même observation tout en contenant un nombre variable de fautes, prenant ainsi en compte les différentes interprétations possibles d'une observation. K_2 , quant à lui, permet de considérer les fautes résiduelles, hors de la portée des séquences considérées par K_1 .

Remarque : La marge d'erreur additive est le cas particulier de la marge multiplicative où $K_1 = 1$.

Le compteur C défini plus tôt serait un compteur additif pour lequel $g(C(\pi_\rho)) = C(\pi_\rho) + 1$, donnant ainsi :

$$C(\pi_\rho) \leq |\rho|_f \leq C(\pi_\rho) + 1$$

2 Temps de détection d'une faute

Nous allons commencer par nous intéresser au temps que met un compteur C pour détecter une faute f donnée. Ici, nous avons fait le choix de ne pas considérer de systèmes temporisés. Le temps est alors exprimé comme un nombre de transitions entre l'apparition d'une faute dans une exécution et sa détection par le compteur. Notons que nous ne nous intéresserons pas dans cette partie à la construction de C ni à ses propriétés mais simplement à sa capacité à détecter une faute au bout d'un certain temps.

2.1 Définitions

Afin de simplifier les prochains raisonnements, nous allons restreindre l'ensemble des systèmes étudiés à ceux représentés par un automate $\mathcal{A} = (Q, \Sigma, \rightarrow, q_0)$ de cette forme :

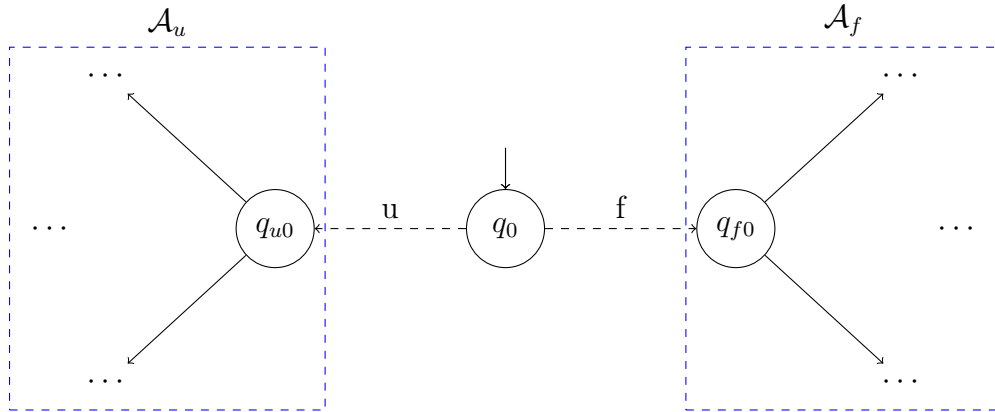


FIGURE 2 – Format des systèmes considérés

Le principe est d'avoir un état initial q_0 qui est le point de départ de l'exécution du système. À partir de cet état, deux transitions sont possibles :

- la transition fautive f qui mène à l'état q_{f0} ,
- la transition non-fautive u qui mène à l'état q_{u0} .

Les états q_{f0} et q_{u0} sont les points de départ de deux sous-automates $\mathcal{A}_f$ et $\mathcal{A}_u$ disjoints. Ce système ne contient qu'une unique transition fautive.

Commençons alors par définir formellement ces deux sous-automates :

Définition 6 (Sous-automates $\mathcal{A}_f$ et $\mathcal{A}_u$). *Nous pouvons définir $\mathcal{A}_f = (Q_f, \Sigma_f, \rightarrow_f, q_{f0})$ tel que :*

- q_{f0} est l'état atteint en partant de q_0 et en suivant la transition fautive,
- Q_f est l'ensemble des états de $\mathcal{A}$ atteignables en passant par la transition (q_0, f, q_{f0}) ,
- Σ_f est l'ensemble des symboles des transitions tels que :
Soit une transition $(q, l, q') \in \rightarrow_f$, $l \in \Sigma_f$ ssi $q \in Q_f$,
- $\rightarrow_f$ est l'ensemble des transitions telles que $(q, l, q') \in \rightarrow_f$ ssi $q \in Q_f$.

Nous noterons que $\mathcal{A}_f$ est non-bloquant, c'est-à-dire que pour tout état de Q_f il est toujours possible d'effectuer une transition menant vers un état de Q_f , et qu'il ne contient aucune faute.

Nous définissons $\mathcal{A}_u = (Q_u, \Sigma_u, \rightarrow_u, q_{u0})$ de la même manière que $\mathcal{A}_f$ mais en remplaçant f par u .

Afin de définir le temps de détection d'une faute, il va être utile de définir la taille des précédents automates :

Notation (Taille des automates). On note N_f la taille de $\mathcal{A}_f$ telle que $N_f = |Q_f|$ et N_u la taille de $\mathcal{A}_u$ telle que $N_u = |Q_u|$. On note N la taille de l'automate $\mathcal{A}$ telle que :

$$N = |Q| = N_f + N_u + 1$$

Définition 7 (Temps de détection d'une faute). On définit t_f le temps de détection de la faute f comme le nombre de transitions minimal nécessaire pour que le compteur C puisse compter la faute f (en considérant que le comptage commence à partir de la première transition partant de q_{f0} ou q_{u0}).

2.2 Temps minimal et maximal

Après avoir défini le temps de détection d'une faute, nous nous sommes posé la question de savoir s'il était possible de lui déterminer une borne inférieure et une borne supérieure. Mais avant de se lancer dans la recherche de ces bornes, il est important de se demander si t_f peut être borné ou non car, dans le cas où la faute f est indétectable, la question de la borne inférieure et de la borne supérieure ne se pose pas et on considère que $t_f = +\infty$.

Il existe en effet des systèmes dans lesquels il est impossible de détecter une faute, même après un nombre infini de transitions. En voici un exemple :

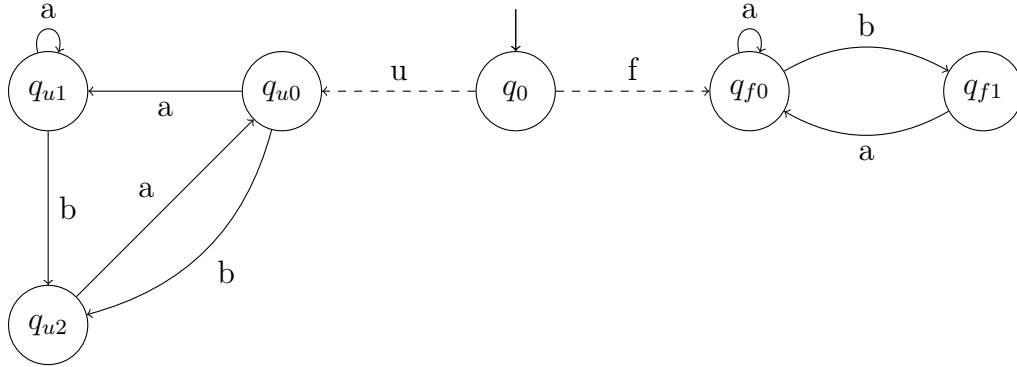


FIGURE 3 – Système avec faute indétectable

Ici, il n'existe aucun moyen de distinguer les exécutions de $\mathcal{A}_f$ de celles de $\mathcal{A}_u$ à partir de l'observation, la faute ne peut donc pas être détectée.

Il faut alors commencer par trouver une méthode permettant de déterminer si la faute f est détectable ou non. Dans le cas de notre système, une méthode possible serait de s'intéresser, pour tout symbole de transition l , si les deux automates $\mathcal{A}_f$ et $\mathcal{A}_u$ peuvent effectuer une transition σ de symbole l simultanément.

- Si oui alors on ne peut pas encore déterminer si l'exécution se déroule dans $\mathcal{A}_f$ ou $\mathcal{A}_u$.
- Si $\mathcal{A}_f$ peut effectuer la transition σ mais pas $\mathcal{A}_u$ alors on peut affirmer que l'exécution se déroule dans $\mathcal{A}_f$ et donc que f a eu lieu.
- Si $\mathcal{A}_u$ peut effectuer la transition σ mais pas $\mathcal{A}_f$ alors on peut affirmer que l'exécution se déroule dans $\mathcal{A}_u$ et donc que f n'a pas eu lieu.

Pour obtenir ce résultat, nous avons eu besoin de créer une opération capable de produire un automate $\mathcal{A}_d$ similaire à l'intersection de $\mathcal{A}_f$ et $\mathcal{A}_u$ mais auquel nous ajouterions des propriétés.

Pour rappel, l'intersection de deux automates $\mathcal{A}_u = (Q_u, \Sigma_u, \rightarrow_u, q_{u0})$ et $\mathcal{A}_f = (Q_f, \Sigma_f, \rightarrow_f, q_{f0})$ est un automate $\mathcal{A}_c = (Q_c, \Sigma_c, \rightarrow_c, q_{c0})$ tel que :

- $Q_c = Q_u \times Q_f$
- $\Sigma_c = \Sigma_u \cup \Sigma_f$
- $\rightarrow_c = \{((q_u, q_f), l, (q'_u, q'_f)) \mid (q_u, l, q'_u) \in \rightarrow_u, (q_f, l, q'_f) \in \rightarrow_f\}$
- $q_{c0} = (q_{u0}, q_{f0})$

En partant de la définition précédente de l'intersection de deux automates $\mathcal{A}_u$ et $\mathcal{A}_f$, nous avons défini une nouvelle opération qui permet de construire un automate $\mathcal{A}_d$ tenant compte des transitions possibles pour l'un des deux automates mais pas l'autre.

Pour cela, nous définissons deux états détecteurs de faute et d'absence de faute d_f et d_u qui correspondent chacun à l'état dans lequel nous nous trouvons lorsque l'un des automates ne peut pas effectuer une transition alors que l'autre en est capable. Cette opération est appelée intersection avec détecteurs et nous la définissons comme suit :

Définition 8 (Intersection avec détecteurs). *On définit $\mathcal{A}_d = (Q_d, \Sigma_d, \rightarrow_d, q_{d0})$ l'intersection avec détecteurs de $\mathcal{A}_u = (Q_u, \Sigma_u, \rightarrow_u, q_{u0})$ et $\mathcal{A}_f = (Q_f, \Sigma_f, \rightarrow_f, q_{f0})$ telle que :*

- $Q_d = Q_u \times Q_f \cup \{d_f, d_u\}$
- $\Sigma_d = \Sigma_u \cup \Sigma_f$
- $q_{d0} = (q_{u0}, q_{f0})$

Avec $\rightarrow_d$ tel que :

- $((q_u, q_f), l, (q'_u, q'_f)) \in \rightarrow_d$ quand $(q_u, l, q'_u) \in \rightarrow_u$ et $(q_f, l, q'_f) \in \rightarrow_f$
- $((q_u, q_f), l, d_f) \in \rightarrow_d$ quand $\exists q'_f, (q_f, l, q'_f) \in \rightarrow_f$ et $\nexists q'_u, (q_u, l, q'_u) \in \rightarrow_u$
- $((q_u, q_f), l, d_u) \in \rightarrow_d$ quand $\nexists q'_f, (q_f, l, q'_f) \in \rightarrow_f$ et $\exists q'_u, (q_u, l, q'_u) \in \rightarrow_u$

Cette nouvelle opération étant définie, nous avons déterminé plusieurs propriétés nous renseignant sur la possibilité de détecter t_f .

Lemme 1 (Non détection de la faute 1). *Si les états d_f et d_u sont inatteignables alors $t_f = +\infty$.*

Démonstration. *Les états détecteurs d_f et d_u sont les états permettant d'affirmer que la faute f a eu lieu ou non. Si ces deux états ne sont pas atteignables alors il est impossible de savoir si la faute a eu lieu ou non, et donc le temps de détection de la faute est infini ($t_f = +\infty$).*

Lemme 2 (Non détection de la faute 2). *Si $\mathcal{A}_d$ contient un cycle alors $t_f = +\infty$.*

Démonstration. *Si $\mathcal{A}_d$ contient un cycle alors il existe une exécution infinie de $\mathcal{A}_d$ passant une infinité de fois par ce cycle. Dans ce cas, les états d_f et d_u ne sont jamais atteints et donc $t_f = +\infty$ (cf. Lemme 1).*

Aussi, en supposant que d_f ne soit pas atteignable mais que d_u le soit alors

nous ne pourrions détecter que les cas où la faute n'a pas eu lieu, donc le calcul de t_f n'aurait pas de sens. Nous pouvons cependant montrer que ce cas ne se produit jamais.

Lemme 3. *Dans le cas où $\mathcal{A}_d$ est acyclique alors d_u et d_f sont atteignables.*

Démonstration. En partant de q_{d0} , on prend une transition de $\mathcal{A}_d$ qui mène possiblement :

1. Dans d_u .
2. Dans d_f .
3. Dans $q_{d0} \rightarrow$ impossible car $\mathcal{A}_d$ est acyclique.
4. Dans un nouvel état $q = (q_a, q_b)$.

Dans le cas 4., on continue à prendre des transitions jusqu'à ce qu'on tombe dans un autre cas ou qu'on retombe sur un état visité. Si on retombe sur un état visité alors il y a un cycle, ce qui est impossible car $\mathcal{A}_d$ est acyclique. On ne peut pas continuer à l'infini car il y a un nombre fini d'états.

Si la transition nous amène dans d_u , comme $\mathcal{A}_f$ est non-bloquant, on peut toujours trouver une transition depuis (q_a, q_b) qui correspond à une transition faisable dans $\mathcal{A}_f$. On tombe donc soit dans le cas 2. (= d_f est atteignable), soit dans le cas 4. (= on repart dans le cas expliqué au paragraphe précédent) (1. n'étant pas possible car une transition faisable dans $\mathcal{A}_f$ ne peut pas donner une transition de $\mathcal{A}_d$ menant dans d_u).

On peut donc affirmer que si d_u est atteignable alors on finira forcément par tomber dans le cas 2. qui implique que d_f est atteignable.

Afin de déterminer dans quel cas de figure on se trouve, il nous faut déterminer si $\mathcal{A}_d$ contient un cycle. Pour cela, il existe un algorithme [2] en temps polynomial dont le principe est le suivant :

- On effectue un parcours en profondeur d'abord partant de l'état q_{d0} .
- On marque chaque état visité.
- Si on visite un état déjà marqué alors on a trouvé un cycle.
- Si on a visité tous les états sans en trouver un déjà marqué alors il n'y a pas de cycle.

Maintenant que l'on a déterminé si la faute est détectable ou non, on peut se pencher sur le calcul des bornes de t_f .

Pour commencer, on peut déterminer la borne inférieure de t_f de manière triviale. En effet, si $t_f = 0$, alors C pourrait compter f dès la première transition, ce qui est impossible car la première transition de toute observation π_ρ est soit f soit u qui sont toutes deux inobservables. Ainsi :

Lemme 4 (Borne inférieure). *On peut poser une borne inférieure pour t_f telle que $0 < t_f$.*

On peut aussi se baser sur le fait que $\mathcal{A}_d$ est acyclique pour affirmer que la

borne supérieure de t_f est donnée par la longueur du plus long chemin de $\mathcal{A}_d$ menant de q_{d0} à d_f , ainsi :

Lemme 5 (Borne supérieure). *Dans le cas où $\mathcal{A}_d$ est acyclique et où d_f est atteignable, on a :*

$$t_f < N_f \times N_u$$

Démonstration. *On peut affirmer que t_f est au plus égal à la longueur du plus long chemin de $\mathcal{A}_d$ allant de q_{d0} à d_f . Or le nombre d'états N_d de $\mathcal{A}_d$ est au plus égal à $N_f \times N_u$. Comme il n'existe pas de cycle, la longueur du plus long chemin est au plus égale à $N_d - 1$. On a donc :*

$$t_f \leq N_d - 1 \leq N_f \times N_u - 1 < N_f \times N_u$$

Pour ce qui est de déterminer le plus long chemin allant de q_{d0} à d_f , il existe un algorithme [3] polynomial dans un graphe orienté acyclique fonctionnant ainsi :

- On effectue un tri topologique sur le graphe.
- On choisit l'état q_{d0} comme point de départ de l'algorithme et on initialise un tableau contenant les plus grandes distances entre q_{d0} et les autres sommets du graphe.
- Pour chaque sommet v dans l'ordre topologique, on met à jour le tableau s'il existe un chemin de u à v (avec u un prédécesseur de v) qui augmente la plus grande distance déjà inscrite dans le tableau.
- L'ordre topologique permet de s'assurer que tous les prédécesseurs de v ont déjà été traités avant d'arriver à v .

À partir des précédents lemmes on peut énoncer le théorème suivant :

Théorème (Bornes de t_f). *Soit $\mathcal{A}_d$ le résultat de l'intersection avec détecteurs de $\mathcal{A}_f$ et $\mathcal{A}_u$. Si $\mathcal{A}_d$ est acyclique et d_f est atteignable alors :*

$$0 < t_f < N_f \times N_u$$

3 Cycles et paire de cycles ambiguë

Lorsque nous représentons des systèmes réels sous forme d'automates, l'utilisation de cycles pour représenter fidèlement leurs comportements est courante. Il est donc normal de demander quels seraient les comportements possibles ou attendus d'une fonction de comptage sur ces automates. Dans cette partie, nous allons voir que sous certaines conditions il n'existe pas de fonction de comptage pouvant répondre à nos besoins.

3.1 Définitions

Définition 9 (Cycle). *Soit un automate $\mathcal{A} = (Q, \Sigma, \rightarrow, q_0)$. Un cycle est une séquence d'états γ_c telle qu'il existe un état q et un entier $k > 0$ vérifiant :*

- $\gamma_c[0] = q$ et $\gamma_c[n] = q$,
- pour tout $i \in \{1, \dots, n\}$, $\gamma_c[i-1] \rightarrow \gamma_c[i]$ est une transition valide de l'automate,
- il existe une décomposition de γ_c en k sous-séquences consécutives :

$$\gamma_c = \gamma^{(1)} \cdot \gamma^{(2)} \cdot \dots \cdot \gamma^{(k)}$$

telles que pour tout $j \in \{1, \dots, k\}$:

- $\gamma^{(j)}[0] = \gamma^{(j)}[n_j] = q$,
- $\forall i \in]0, n_j[, \gamma^{(j)}[i] \neq q$.

On dit alors que γ_c est un cycle effectuant k tours complets autour de l'état q .

La comparaison entre cycles dans un automate est une chose que nous devons prendre en compte si nous voulons définir des fonctions de comptage adaptées et les plus précises possible. Afin de démontrer certains points, nous avons défini des notions telles que la distinguabilité de deux séquences, la co-accessibilité des cycles ou la concurrence entre cycles.

Définition 10 (Distinguabilité). *Deux séquences γ_1 et γ_2 sont dites distinguables si et seulement si elles ont des observations différentes, c'est-à-dire si et seulement si $\pi_{\gamma_1} \neq \pi_{\gamma_2}$.*

Définition 11 (Coaccessibilité). *Deux cycles γ_{c_1} et γ_{c_2} sont dits coaccessibles si et seulement s'ils peuvent être atteints à partir de deux séquences γ_1 et γ_2 indistinguables.*

Définition 12 (Concurrence). *Soient deux cycles γ_{c_1} et γ_{c_2} . On dit que ces cycles forment une paire de cycles concurrents si :*

- ils sont indistinguables,
- ils sont coaccessibles,
- ils produisent un nombre différent de fautes, c'est-à-dire $|\gamma_{c_1}|_f \neq |\gamma_{c_2}|_f$.

Définition 13 (Paire de cycles ambiguë). Soient deux cycles γ_{c_1} et γ_{c_2} . La paire de cycles $(\gamma_{c_1}, \gamma_{c_2})$ est dite ambiguë si :

- les deux cycles $(\gamma_{c_1}$ et $\gamma_{c_2})$ sont concurrents,
- γ_{c_1} ne contient aucune transition fautive.

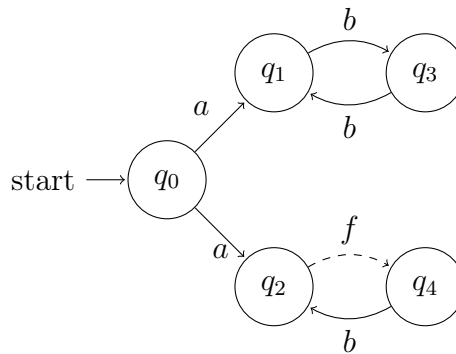
Remarque. Comme les deux cycles sont concurrents et que γ_{c_1} ne contient aucune transition fautive, γ_{c_2} en contient au minimum une.

3.2 Paire de cycles ambiguë et fonction de comptage

Théorème. Soit un système possédant une paire de cycles ambiguë. Il est impossible de définir une fonction de comptage des fautes C .

L'automate ci-dessous est un exemple d'automate dans lequel se trouve une paire de cycles ambiguë. Nous pourrions nous appuyer sur cet exemple pour montrer l'intuition qui nous a menée à ce théorème puis le prouver formellement avec une démonstration.

Exemple d'automate avec une paire de cycles ambiguë



Séquences d'accès :

- γ_1 : séquence d'accès vers q_1 : $q_0 \xrightarrow{a} q_1$,
- γ_2 : séquence d'accès vers q_2 : $q_0 \xrightarrow{a} q_2$.

Cycles :

- γ_{c_1} : cycle autour de q_1 : $q_1 \xrightarrow{b} q_3 \xrightarrow{b} q_1$,
- γ_{c_2} : cycle autour de q_2 (2 tours) : $q_2 \xrightarrow{f} q_4 \xrightarrow{b} q_2 \xrightarrow{f} q_4 \xrightarrow{b} q_2$.

Les cycles γ_{c_1} et γ_{c_2} sont bien coaccessibles : γ_1 et γ_2 produisent la même observation a et lorsque l'on obtient une observation bb , le nombre de faute de γ_{c_1} est de 0 tandis que celui de γ_{c_2} est de 2. On ne peut donc pas savoir dans quel cycle on se trouve en regardant seulement l'observation. Le nombre de fautes est donc impossible à compter lorsque le nombre de tours de cycle tend vers l'infini.

Démonstration. Soit un automate. Supposons qu'il existe une paire de cycles ambiguë $\gamma_{c_1}, \gamma_{c_2}$. Par définition, il existe également des séquences γ_1 et γ_2 . Supposons qu'il existe aussi une fonction de comptage C , il existe donc une fonction positive quelconque $g : \mathbb{N} \rightarrow \mathbb{N}$ pour laquelle on a : $|\rho|_f \leq g(C(\pi_\rho))$.

Comme γ_1 et γ_2 sont indistinguables, nous avons : $C(\pi_{\gamma_1}) = C(\pi_{\gamma_2}) \leq \min(|\gamma_1|_f, |\gamma_2|_f)$

Après m itérations des cycles, le nombre de fautes dans chaque exécution est donné par :

$$\begin{aligned} |\gamma_1 \cdot \gamma_{c_1}^m|_f &= |\gamma_1|_f \\ |\gamma_2 \cdot \gamma_{c_2}^m|_f &= |\gamma_2|_f + m \cdot |\gamma_{c_2}|_f \end{aligned}$$

Nous définissons l'exécution suivante : $\rho_m = \gamma_2 \cdot \gamma_{c_2}^m$

La fonction de comptage C satisfait donc la condition suivante : $|\rho_m|_f \leq g(C(\pi_{\rho_m}))$

Pour garantir la validité de la fonction g , nous supposons qu'elle est définie sur l'intervalle $[0, |\gamma_1|_f]$ et qu'elle est bornée par une constante α , où :

$$\alpha = \max_{i \in [0, |\gamma_1|_f]} g(i)$$

D'après les conditions imposées sur γ_1 , nous avons aussi :

$$C(\pi_{\rho_m}) = C(\pi_{\gamma_1 \cdot \gamma_{c_1}^m}) \leq |\gamma_1|_f$$

d'où

$$g(C(\pi_{\rho_m})) \leq \alpha$$

Cependant, lorsque $m \rightarrow \infty$, le nombre de fautes dans ρ_m devient :

$$|\rho_m|_f = |\gamma_2|_f + m \cdot |\gamma_{c_2}|_f$$

Or, comme $|\gamma_{c_2}|_f > 0$, l'inégalité suivante est impossible :

$$|\rho_m|_f = |\gamma_2|_f + m \times |\gamma_{c_2}|_f \leq \alpha$$

En effet, le nombre de fautes dans ρ_m tend vers l'infini tandis qu' α est une constante. Cela contredit la satisfaction de la condition de la fonction de comptage C car $g(C(\pi_{\rho_m}))$ est bornée par la constante α .

Il est donc impossible qu'une telle fonction g puisse satisfaire la condition imposée pour une fonction de comptage C en présence d'une paire de cycles ambiguë.

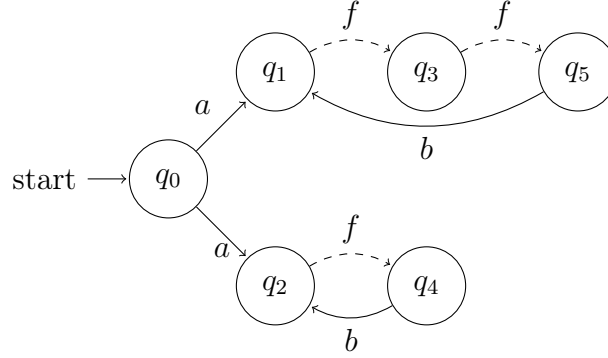
□

3.3 Cycles concurrents et fonction de comptage additive

Théorème. Soit un automate dans lequel se trouve une paire de cycles concurrents $(\gamma_{c_1}, \gamma_{c_2})$. Il est impossible de définir une fonction de comptage additive des fautes C .

Ce théorème ressemble au théorème précédent mais il prend en compte des cycles concurrents quelconques et ne dit que seule les fonctions de comptage additives sont impossibles à utiliser. Nous pourrions nous appuyer sur l'exemple ci-dessous pour montrer l'intuition avant de prouver formellement ce théorème.

Exemple d'automate avec une paire de cycles concurrents



Séquences d'accès :

- γ_1 : séquence d'accès vers q_1 : $q_0 \xrightarrow{a} q_1$,
- γ_2 : séquence d'accès vers q_2 : $q_0 \xrightarrow{a} q_2$.

Cycles :

- γ_{c_1} : cycle autour de q_1 : $q_1 \xrightarrow{f} q_3 \xrightarrow{f} q_5 \xrightarrow{b} q_1$,
- γ_{c_2} : cycle autour de q_2 : $q_2 \xrightarrow{f} q_4 \xrightarrow{b} q_2$.

Les cycles γ_{c_1} et γ_{c_2} sont bien coaccessibles : γ_1 et γ_2 produisent la même observation a et on a :

$$|\gamma_{c_1}|_f = 2 \quad \text{et} \quad |\gamma_{c_2}|_f = 1$$

On peut donc compter une faute à chaque b observé. Il existe alors une fonction de comptage multiplicative qui, sur cet exemple pourrait être de la forme :

$$C(\pi_\rho) \leq |\rho|_f \leq K_1 \times C(\pi_\rho) + K_2$$

avec $K_1 = 2$, $K_2 = 2$ et $C(\pi_\rho)$ qui compte 1 faute pour chaque b dans π_ρ .

Démonstration. *Idée : les nombres de fautes dans les deux cycles vont croître à des vitesses différentes car ils ont un nombre différent de transitions fautives. En particulier, lorsque le nombre de tours de cycle devient grand, la différence du nombre de transitions fautives entre les deux cycles devient arbitrairement grande, alors que l'égalité entre les fautes réellement comptées reste, ce qui entraîne une violation de la marge d'erreur additive supposée.*

Le cas où l'un des deux cycles ne contient pas de transition fautive a déjà été

traité dans la partie précédente concernant la présence d'une paire de cycles **ambiguë** dans un automate.

Supposons maintenant par l'absurde qu'il existe une fonction de comptage additive des fautes C définie sur les exécutions, telle que pour toute exécution ρ on ait :

$$C(\pi_\rho) = N \leq |\rho|_f \leq N + K$$

Les deux cycles sont concurrents, on peut donc considérer deux séquences d'accès γ_1 et γ_2 , respectivement vers les cycles γ_{c_1} et γ_{c_2} . Considérons les exécutions suivantes, pour tout entier $m \geq 0$:

$$\rho_m^1 = \gamma_1 \cdot \gamma_{c_1}^m$$

$$\rho_m^2 = \gamma_2 \cdot \gamma_{c_2}^m$$

Comme les cycles sont concurrents, on a $\pi_{\rho_m^1} = \pi_{\rho_m^2}$, et on note :

$$C(\pi_{\rho_m^1}) = C(\pi_{\rho_m^2}) = N_m$$

On note aussi d la différence absolue de transitions fautives entre ρ_m^1 et ρ_m^2 . Or, comme les deux cycles sont concurrents, ρ_m^2 contient au moins une transition fautive de plus que ρ_m^1 , on a donc $d \geq 1$. La fonction de comptage étant additive, on a aussi :

$$N_m \leq |\rho_m^1|_f \leq N_m + K \quad \text{et} \quad N_m \leq |\rho_m^2|_f \leq N_m + K$$

Comme $d \geq 1$, on a :

$$(K + 1) \times d \geq K + 1 > K$$

Choisissons maintenant m tel que $m = K + 1$. On obtient alors :

$$N_m \leq |\rho_m^1|_f + (K + 1) \times d \leq N_m + K$$

Cette inégalité est absurde. L'hypothèse initiale est donc fausse : il est impossible de définir une fonction de comptage additive des fautes dans un automate comportant une paire de cycles concurrents.

□

Ces deux théorèmes utilisent le fait qu'on connaît les propriétés des cycles pris en compte (nombre d'états, de transitions fautives, position dans l'automate, etc.). Il serait intéressant d'avoir un moyen de détecter une paire de cycle ambiguë dans un automate ou de savoir si d'autres contextes empêcheraient l'utilisation de fonctions de comptage. Cette piste n'est pas à omettre car elle permettrait de mieux caractériser les fonctions de comptage et de justifier leur utilisation dans notre contexte.

4 Construction par recouvrement de compteurs approchés

Après avoir posé les bases formelles des compteurs approchés dans la partie 1, cette dernière partie s'attache à présenter une méthode concrète de construction. D'autres approches restent envisageables, chacune reposant sur une interprétation particulière de la structure d'un système.

La construction choisie dans ce rapport repose sur le recouvrement d'une observation par des fragments plus petits appelés segments. S'il est établie que certains segments sont les résultats de séquences pour lesquelles il se produit systématiquement au moins un faute, alors la présence de ces segments dans l'observation d'une exécution peut être exploitée pour l'estimation du nombre de fautes dans cette exécution.

4.1 Cadre formel

On définit formellement les segments comme des sous-ensembles d'une observation, et leurs notations à partir d'une observation.

Définition 14 (Segment). *s est un segment de π_ρ s'il existe h et t tel que $h \cdot s \cdot t = \pi_\rho$.*

Notation. *Pour une observation $\pi_\rho = a_1 a_2 \dots a_{n-1} a_n$*

Le segment $s = a_1$ est noté $\pi_\rho(1)$.

Le segment $s' = a_2 \dots a_{n-1}$ est noté $\pi_\rho(2, n-1)$.

Pour cette construction, on considère exclusivement des segments non-chevauchants, c'est-à-dire des segments qui se succèdent sans se superposer. De cette façon à ce que chaque observable appartient à un, et un seul, de ces segments.

Définition 15 (Segments non-chevauchants). *s et s' des segments distincts de π_ρ sont non-chevauchants si et seulement si pour $s = \pi_\rho(i_1, i_2)$ et $s' = \pi_\rho(j_1, j_2)$, on a $i_2 < j_1$ ou $j_2 < i_1$.*

Pour cette construction, on considère une observation π_ρ comme une partition de segments non-chevauchants : $\pi_\rho = s_1 \cdot s_2 \cdot \dots \cdot s_n$

On suppose qu'on dispose d'une analyse a priori du système à partir duquel, on peut construire S_f l'ensemble des segments ne pouvant être obtenus que

si au moins une faute s'est produite pendant la séquence engendrant le segment, leur présence dans un recouvrement infère donc l'existence d'au moins une faute supplémentaire dans l'exécution considérée. Par construction, on supposera que S_f ne contient que des segments minimaux, c'est-à-dire que s'il existe des séquences d'observations r , h et t tels que $s = r \cdot h \cdot t$, alors h appartient à S_f implique que s n'appartient pas à S_f .

Dans le cas du système illustré dans la figure 1, l'analyse identifie les séquences suivantes :

- | | |
|--|--|
| 1. $q_1 \xrightarrow{c} q_2 \xrightarrow{f} q_0 \xrightarrow{a} q_1$ | 5. $q_2 \xrightarrow{e} q_4 \xrightarrow{f} q_0 \xrightarrow{c} q_5$ |
| 2. $q_1 \xrightarrow{c} q_2 \xrightarrow{f} q_0 \xrightarrow{c} q_5$ | 6. $q_1 \xrightarrow{d} q_3 \xrightarrow{u} q_4 \xrightarrow{f} q_0 \xrightarrow{a} q_1$ |
| 3. $q_0 \xrightarrow{c} q_5 \xrightarrow{f} q_6 \xrightarrow{a} q_0$ | 7. $q_1 \xrightarrow{d} q_3 \xrightarrow{u} q_4 \xrightarrow{f} q_0 \xrightarrow{c} q_5$ |
| 4. $q_2 \xrightarrow{e} q_4 \xrightarrow{f} q_0 \xrightarrow{a} q_1$ | |

À partir des observations des ces séquences, l'analyse produit l'ensemble : $S_f = \{ca, cc, ea, ec, da, dc\}$

On construit par la suite le recouvrement $\mathcal{R} = (s_1, \dots, s_n)$ de π_ρ .

À partir de cette construction on peut obtenir une minoration du nombre de fautes de ρ par $C(\pi_\rho) = |\{\{s_k \mid s_k \in S_f \text{ et } s_k \in \mathcal{R}\}\}| = N$. De cette façon, on obtient bien :

$$N \leq |\rho|_f$$

Pour limiter le nombre de recouvrements, on concaténera les segments de $\mathcal{R}$ contigus qui n'appartiennent pas à S_f .

À partir du S_f obtenu pour le système de la figure 1, on peut calculer un recouvrement $\mathcal{R} = (adbac, ea)$ de l'observation $\pi_\rho = adbaceea$, d'où $C(\pi_\rho) = |\{\{ea\}\}| = 1$

4.2 Exemples d'application

Construisons des recouvrements efficaces pour quelques systèmes simples.

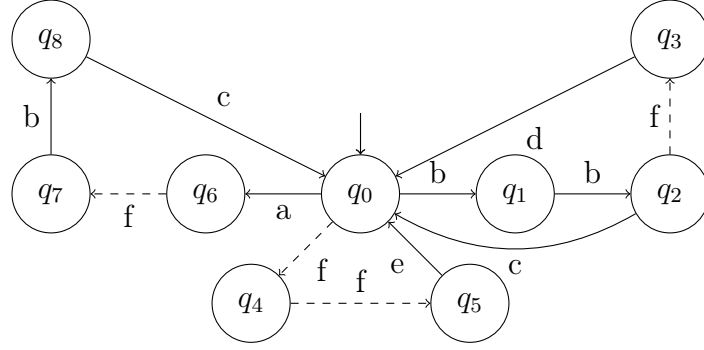


FIGURE 4 – Système supportant des compteurs additifs

À partir de l'analyse on obtient $S_f = \{e, ab, bd\}$.

Pour le segment a , une faute par apparition est considérée, toutefois le système est tel qu'il reste possible de manquer au plus une faute pour ce segment, on considère donc $K = 1$

Pour la trace $\tau_\rho = afbcbbfdbbcf$, on a :

- $|\rho|_f = 3$,
- l'observation $\pi_\rho = abcbbdbbc$

Cette observation admet le recouvrement suivant $\mathcal{R} = (ab, cb, bd, bbc)$. Parmi les segments présents dans $\mathcal{R}$, seuls les segments ab et bd sont dans S_f on a donc :

$$C(\pi_\rho) = 2 \leq |\rho|_f = 3$$

Construit de cette façon, le compteur a la propriété d'être additif avec $K = 2$, satisfaisant :

$$|\rho|_f = 3 \leq C(\pi_\rho) + K = 4$$

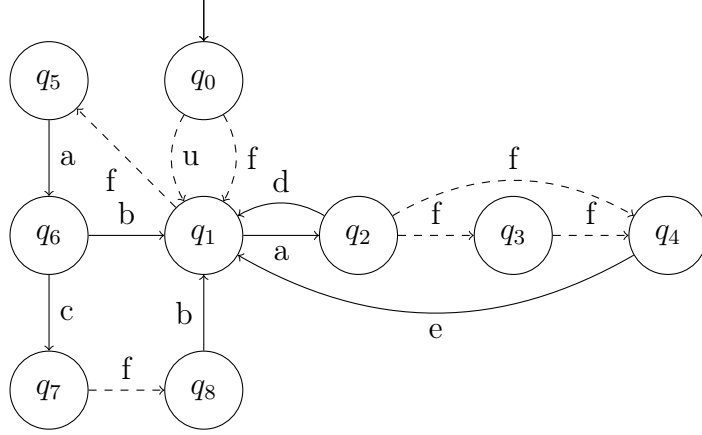


FIGURE 5 – Système supportant des compteurs multiplicatifs

À partir de l'analyse on obtient $S_f = \{ab, ac, cb, ae\}$

Pour la trace $\tau_\rho = ufab(af fe)^n afeadaff$, avec $n \in \mathbb{N}$, on a :

- $|\rho|_f = 2n + 4$,
- l'observation $\pi_\rho = ab(ae)^{n+1}ada$

Cette observation admet le recouvrement suivant $\mathcal{R} = (ab, \underbrace{ae, \dots, ae}_{n+1 \text{ fois}}, ada)$,

le segment ab et les $n + 1$ segments ae de $\mathcal{R}$ étant des segments de S_f , on obtient l'estimation $C(\pi_\rho) = n + 2$, on a donc :

$$C(\pi_\rho) = n + 2 \leq |\rho|_f = 2n + 4$$

Construit de cette façon, le compteur a la propriété d'être multiplicatif avec $K_1 = 2$ et $K_2 = 3$, satisfaisant :

$$|\rho|_f = 2n + 4 \leq K_1 \times C(\pi_\rho) + K_2 = 2n + 7$$

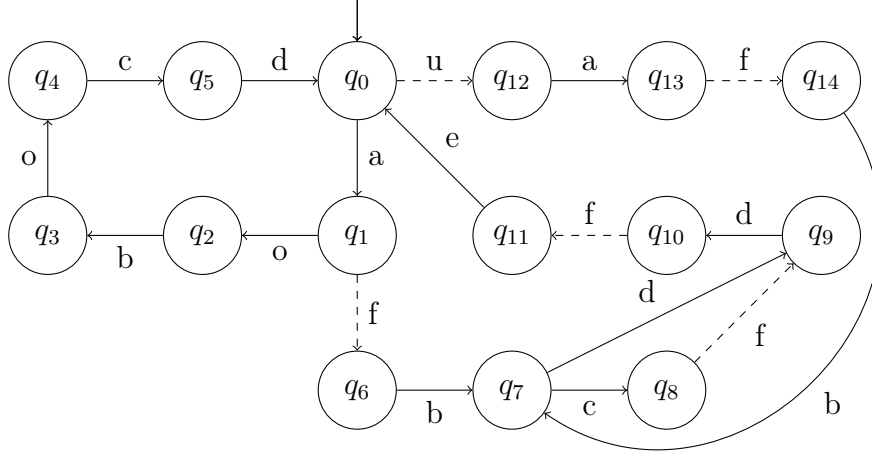


FIGURE 6 – Système d'observations à recouvrements multiples

À partir de l'analyse on obtient $S_f = \{ab, bcd, de\}$

Pour la trace $\tau_\rho = (uafbcfdfe)^2af$, on a :

- $|\rho|_f = 7$,
- l'observation $\pi_\rho = (abcde)^2a$

On peut calculer certains des recouvrements possibles de π_ρ :

- $\mathcal{R}_1 = (ab, c, de, ab, c, de, a)$,
- $\mathcal{R}_2 = (a, bcd, ea, bcd, ea)$,
- etc.

À partir de $\mathcal{R}_1$, on obtiendrait $C(\pi_\rho) = 4$, contre $C(\pi_\rho) = 2$ pour $\mathcal{R}_2$.

Construit de cette façon, le compteur a la propriété d'être multiplicatif avec $K_1 = 3$ et $K_2 = 1$, satisfaisant :

- Pour $\mathcal{R}_1$,

$$C(\pi_\rho) = 4 \leq |\rho|_f = 7 \leq K_1 \times C(\pi_\rho) + K_2 = 3 * 4 + 1$$

- Pour $\mathcal{R}_2$,

$$C(\pi_\rho) = 2 \leq |\rho|_f = 7 \leq K_1 \times C(\pi_\rho) + K_2 = 3 * 2 + 1$$

4.3 Limites

4.3.1 Optimalité du recouvrement

Comme vu dans l'exemple précédent, sur le système de la figure 6, une même observation peut donner lieu à plusieurs recouvrements donnant chacun une estimation différentes. Ainsi, s'il y a plusieurs recouvrements, en fonction du recouvrement considéré, une estimation plus ou moins bonne du nombre de fautes peut être faite.

Afin d'identifier, pour une observation donnée, le recouvrement fournissant la meilleure estimation du nombre de fautes, on peut considérer différentes approches : l'utilisation d'un automate construit à partir des éléments de S_f pour qui il est possible d'utiliser un parcours des différents recouvrements. Pour ce faire, on propose la construction suivante à partir de S_f .

Approche par automate

Avec cette approche, on construit donc un automate à partir des éléments de S_f . Cet automate lit ensuite une observation pour produire une fonction de pondération qui donne le nombre de segments de S_f lus, et donc présents dans l'observation.

Soit $\mathcal{A}_{\mathcal{R}} = (Q_{\mathcal{R}}, \Sigma_{\mathcal{R}}, \rightarrow_{\mathcal{R}}, q_0, F_{\mathcal{R}}, w_{\mathcal{R}})$, tel que :

- $Q_{\mathcal{R}} = \{q_p \mid p \in \text{Pref}_{\mathcal{R}} \setminus S_f\}$ avec $\text{Pref}_{\mathcal{R}} = \bigcup_{s \in S_f} \text{Pref}(s)$: l'ensemble des états encodant les préfixes des segments de S_f ,
- $q_0 = q_{\epsilon}$: l'état initial,
- $\Sigma_{\mathcal{R}} \subseteq \Sigma_o$,
- $\rightarrow_{\mathcal{R}}$: l'ensemble des transitions,
 $\forall q_p \in Q_{\mathcal{R}}, \forall a \in \Sigma_o$, si $p \cdot a \in \text{Pref}_{\mathcal{R}}$, on définit :
 - $q_p \xrightarrow{a} q_0$, si $p \cdot a \in S_f$,
 - $q_p \xrightarrow{a} q_{p \cdot a}$, sinon.
 sinon :
 - $q_p \xrightarrow{a} q$, avec $q_0 \xrightarrow{a} q$, si $q_a \in Q_{\mathcal{R}}$
 - $q_p \xrightarrow{a} q_0$ sinon
 et $\forall a \in \Sigma_o, q_a \notin Q_{\mathcal{R}}$
 - $q_0 \xrightarrow{a} q_0$
- $F_{\mathcal{R}} = Q_{\mathcal{R}}$: l'ensemble des états acceptants,
- $w_{\mathcal{R}}$: une fonction de pondération qui à partir d'une transition de $\rightarrow_{\mathcal{R}}$

retourne un entier représentant le poids attribué à la transition.

$$w_{\mathcal{R}} = \begin{cases} 1 & \forall (q_p, a, q), a \in \Sigma_o, q_p \in Q_{\mathcal{R}}, \text{ et } p \cdot a \in S_f \text{ ou } a \in S_f \\ 0 & \text{sinon} \end{cases}$$

Il est possible d'étendre $w_{\mathcal{R}}$, qui considère des transitions, à des exécutions ; ce qui, pour une exécution $\rho_{\mathcal{R}} = q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \rightarrow \dots \xrightarrow{a_{n-1}} q_n$ donne :

$$W_{\mathcal{R}}(\rho_{\mathcal{R}}) = \sum_{i=1}^n w(q_{i-1} \xrightarrow{a_i} q_i)$$

Pour l'observation $\pi_{\rho} = abcde$ du système de la figure 6, l'utilisation de cette construction permet d'obtenir le résultat suivant :

$$\begin{aligned} - \rho_{\mathcal{R}} &= q_{\epsilon} \xrightarrow{a} q_a \xrightarrow{b} q_{\epsilon} \xrightarrow{c} q_{\epsilon} \xrightarrow{d} q_d \xrightarrow{e} q_{\epsilon} \\ - W_{\mathcal{R}}(\rho_{\mathcal{R}}) &= w(q_{\epsilon} \xrightarrow{a} q_a) + w(q_a \xrightarrow{b} q_{\epsilon}) + w(q_{\epsilon} \xrightarrow{c} q_{\epsilon}) + w(q_{\epsilon} \xrightarrow{d} q_d) + \\ &\quad w(q_d \xrightarrow{e} q_{\epsilon}) = 0 + 1 + 0 + 0 + 1 = 2 \end{aligned}$$

De cette manière, on trouve $C(\pi_{\rho}) = W_{\mathcal{R}}(\rho_{\mathcal{R}}) = 2$.

Approche par algorithme

Si l'on cherche à obtenir la meilleure estimation possible, sans disposer explicitement du recouvrement qui permet son obtention, l'algorithme qui suit

le permet sans qu'une construction supplémentaire soit nécessaire.

Entrées:

- π_ρ : l'observation associée à l'exécution ρ d'un système $\mathcal{A}$
- S_f : l'ensemble des segments fautifs dans un système $\mathcal{A}$

Sorties:

- $C(\pi_\rho)$: une estimation du nombre de faute dans ρ

Variables:

- n : entier = $|\rho|_f$
- $Fautes$: tableau de $n+1$ entiers initialisés à 0. Un entier à l'indice i correspond au nombre de fautes qu'induit le recouvrement du préfixe de taille i de π_ρ .

DÉBUT

Pour i *allant de 1 à n* **faire**

$Fautes[i] \leftarrow Fautes[i-1]$;

Pour j *allant de 0 à i* **faire**

$segment \leftarrow \pi_\rho(j, i)$;

Si $segment \in S_f$ **alors**

$Fautes[i] \leftarrow \max(Fautes[i], Fautes[j] + 1)$;

finSi

finPour

finPour

retourner $C(\pi_\rho) \leftarrow Fautes[n]$;

FIN

Algorithme 1: Estimation du nombre de fautes dans une exécution ρ par recherche du recouvrement de poids maximal

Analyse de la complexité

Pour une observation de taille n .

La complexité temporelle de l'algorithme repose principalement sur la présence des boucles sur i et j :

- la boucle externe allant de 1 à n s'exécute n fois,
- pour chaque i , la boucle interne itère jusqu'à i valeurs de j .

Le nombre d'itérations total dans la boucle interne sur une exécution complète est :

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \in O(n^2)$$

Chaque itération comporte : le calcul d'un préfixe d'une observation, opération qu'on peut considérer en $O(1)$ s'il n'y a pas de reconstruction, et un test d'appartenance de ce préfixe à un ensemble de k éléments, qui avec une structure de données dédiée, comme une table de hachage, peut être estimé en $O(1)$ ou en $O(k)$ en l'absence d'une telle structure ; chacune sera alors de complexité $O(1)$ ou $O(k)$. Pour $O(n^2)$ itérations de complexité $O(1)$ ou $O(k)$, on trouve une complexité temporelle de $O(n^2)$ ou $O(k \cdot n^2)$.

La complexité spatiale provient du tableau de taille $n + 1$ de nos estimations, elle est donc en $O(n)$.

Optimalité de l'approche gloutonne

On peut remarquer que les recouvrements considèrent un plus grand nombre de fautes quand ils favorisent les segments de S_f détectés dans l'observation et les plus courts possibles. On peut donc supposer qu'une approche gloutonne permettrait d'obtenir la solution optimale de notre problème.

On redéfinit notre problème comme suit ; à partir de :

- une observation $\pi_\rho \in \Sigma_o^*$,
- un ensemble de segments fautifs $S_f \subseteq \Sigma_o^+$

on veut obtenir un recouvrement $\mathcal{R}$, constitué de segments non-chevauchants de π_ρ , qui maximise le nombre n de segments fautifs de S_f dans $\mathcal{R}$.

Une approche gloutonne qui à chaque étape réalise le choix optimal local et qui amène à une solution globale optimale procédera ainsi :

1. À partir du premier observable de π_ρ (de position 0),
2. À chaque position, on recherche le premier segment fautif de S_f , c'est-à-dire le segment $s \in S_f$ le plus court dans π_ρ à partir de la position courante.
3. Si un segment s est trouvé, on comptabilise une faute supplémentaire, on se place à la position suivant le dernier observable de s et on répète la procédure,
4. Sinon on ajoute l'observable à la position courante comme segment dans $\mathcal{R}$, on passe à la position suivante et on répète la procédure.
5. On concatène les segments fautifs contigus.

Théorème. *Pour ce problème de recouvrement, un algorithme glouton renvoie le nombre optimal de fautes.*

Preuve

Soient deux recouvrements d'une observation :

- $\mathcal{R}_G = (g_1, g_2, \dots, g_m)$, Le meilleur recouvrement choisit par l'algorithme glouton.
- $\mathcal{R}_O = (o_1, o_2, \dots, o_k)$. Son recouvrement optimal.

On suppose que le nombre de segments fautifs f_O dans $\mathcal{R}_O$ est plus important que le nombre de segments fautifs f_G dans $\mathcal{R}_G$.

Soient $g_1, \dots, g_k$, les k premiers segments en commun dans $\mathcal{R}_G$ et $\mathcal{R}_O$, et soit i l'indice du premier segment où $\mathcal{R}_G$ et $\mathcal{R}_O$ diffèrent : g_i dans $\mathcal{R}_G$ et o_i dans $\mathcal{R}_O$.

Cas n°1

Si $o_i \notin S_f$, alors il s'agit d'un segment non fautif; la construction de $\mathcal{R}_G$ est telle que si un ou plusieurs segments fautifs commencent à la position i le plus court est considéré, il n'en existe alors pas, g_i est donc issu de la concaténation d'observables non fautifs contigus.

Comme o_i n'est pas fautif et comme g_i a concaténé le plus d'observables non fautifs contigus possible, soit $o_i = g_i$, soit g_i est plus long que o_i .

Quelque soit le cas de figure, comme la construction $\mathcal{R}_G$ ne permet pas de rater un segment fautif alors $f_O \leq f_G$, ce qui est une contradiction.

Cas n°2

Si $o_i \in S_f$, alors il s'agit d'un segment fautif, qui commence à la même position que g_i , par construction $g_i \in S_f$ et il est le plus court possible donc o_i est plus long que nécessaire.

On remplace dans $\mathcal{R}_O$, o_i par g_i , en décalant le reste des segments de $\mathcal{R}_O$ pour convenir au nouvel alignement avec $\mathcal{R}_G$, de cette manière soit il est possible de considérer plus de segments fautifs dans le futur, soit le total de segments fautifs reste le même.

Ainsi, on peut donc remplacer le recouvrement optimal $\mathcal{R}_O$ par un recouvre-

ment $\mathcal{R}_{O'}$ tel que $f_O \leq f_{O'}$, et qui partage un plus grand nombre de segments en commun en début de recouvrement avec $\mathcal{R}_G$.

En réalisant cet échange de manière inductive, on finit par construire une version de $\mathcal{R}_O$ identique à $\mathcal{R}_G$ et qui considère au moins autant de segments fautifs que celui-ci, ce qui est une contradiction.

Ainsi, un recouvrement gloutin qui sélectionne les segments fautifs les plus courts, ou à défaut des observables non fautifs contigus concaténés, produit un recouvrement avec un nombre de fautes maximal.

4.3.2 Dégradation de l'estimation

Avec cette construction, les segments qui recouvrent une observation sont considérés indépendamment les uns des autres, en fonction des systèmes, il se peut que des fautes puissent ne pas être comptées en raison de la forme des segments qui les produisent.

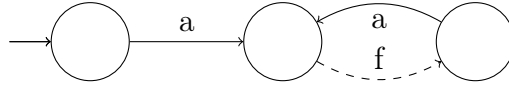


FIGURE 7 – Cas d'estimation dégradée

Dans ce cas, une faute pourra être détectée par le segment aa , ce qui entraîne une estimation deux fois plus petite que le nombre réel de fautes dans les exécutions ce système.

De pires cas de figure peuvent être envisagés.

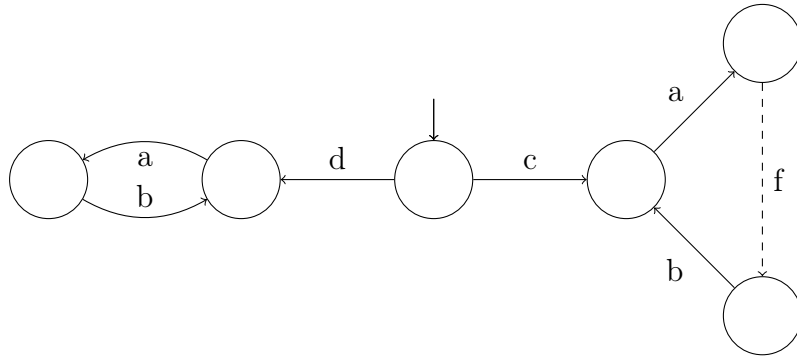


FIGURE 8 – Cas d'estimation constante

Ici, le seul segment fautif trouvé sera : cab . Ainsi des observations de la forme $c(ab)^n$ verraient leur nombre de fautes systématiquement estimées à 1, estimation de plus en plus imprécise à mesure que n augmente.

Quand de tels motifs apparaissent dans un système, il serait préférable de considérer d'autres constructions de compteurs.

5 Conclusion

Ce rapport s'est attaché à formaliser la notion de dégradation d'un système et de sa mesure au travers d'un cadre rigoureux basé sur des systèmes de transitions et sur l'analyse des fautes inobservables par l'intermédiaire d'observations partielles de leurs comportements. Dans un premier temps, nous avons introduit les outils de représentation utiles, ainsi que différentes fonctions de comptage permettant d'estimer le nombre de fautes à partir d'une exécution. La base actuelle pourrait ouvrir la voie à l'exploration d'autres modèles pour la représentation de systèmes, notamment temporels, probabilistes ou intégrant des processus de récupération de fautes, mais aussi à l'étude des propriétés des compteurs comme l'identification systématique des constantes pour les marges, ou l'optimalité d'un type de compteur pour un système donné.

Nous avons ensuite abordé la question du temps de détection d'une faute, en considérant le délai entre la survenue d'une faute et sa détectabilité à partir de l'observation. Cette analyse nous a permis de mettre en lumière des enjeux de réactivité dans les systèmes partiellement observables. Toutefois les résultats présentés utilisent une structure particulière de système, donné dans la figure 2. Une perspective logique serait donc d'examiner dans quelle mesure ces résultats peuvent être généralisées à des systèmes plus complexes ou adaptés pour un cadre plus large. De plus, la borne supérieure proposée pour t_f repose sur la taille du plus long chemin dans $\mathcal{A}_d$. Il serait pertinent d'explorer des méthodes alternatives permettant d'améliorer la précision de cette borne.

La troisième partie s'est concentrée sur l'étude de cycles et de paires de cycles ambiguës, identifiés comme des obstacles potentiels à la mesure de la dégradation. Cette analyse nous a permis de mieux comprendre les limites inhérentes à certains compteurs ainsi qu'à la mesure de la dégradation en général, en suggérant des cas concrets où les compteurs peuvent perdre en précision voire ne pas pouvoir être exploités. Cette partie peut être développée, notamment sur la détection de cycles ou de paires de cycles ambiguës dans un automate. Les pistes recherchées font partie des prémisses du domaine mais servent à définir plus précisément les fonctions de comptage en mettant en avant les limites mais aussi en donnant des cas spécifiques d'utilisation.

Enfin, une méthode concrète de construction de compteurs approchés a été développée, utilisant le recouvrement d'observations par des segments fautifs. En réponse à l'une des limites identifiées de la construction, une construction

d'automate ainsi qu'un algorithme associé ont été proposés pour identifier, parmi tous les recouvrements possibles, celui qui fournit la meilleure estimation. Ces travaux pourraient être approfondis par une méthode systématique pour l'identification des segments fautifs qu'on suppose existante à l'heure actuelle ainsi que par l'étude d'autres constructions s'appuyant par exemple sur des considérations d'un contexte dans les observations.

En conclusion, ce travail propose un cadre formel pour modéliser et mesurer la dégradation dans des systèmes dont les comportements sont partiellement observables. Aussi, il ouvre des perspectives théoriques de poursuites pour le développement d'outils adéquats à la mesure de la dégradation.

Références

- [1] Nathalie Bertrand, Serge Haddad, Engel Lefauchaux. *A Tale of Two Diagnoses in Probabilistic Systems*, Information and Computation, 2019, 269, pp.1-33.
- [2] Robert Tarjan, *Depth-First Search and Linear Graph Algorithms*, SIAM Journal on Computing, 1972.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. 3rd edition, The MIT Press, Cambridge, MA, 2009.